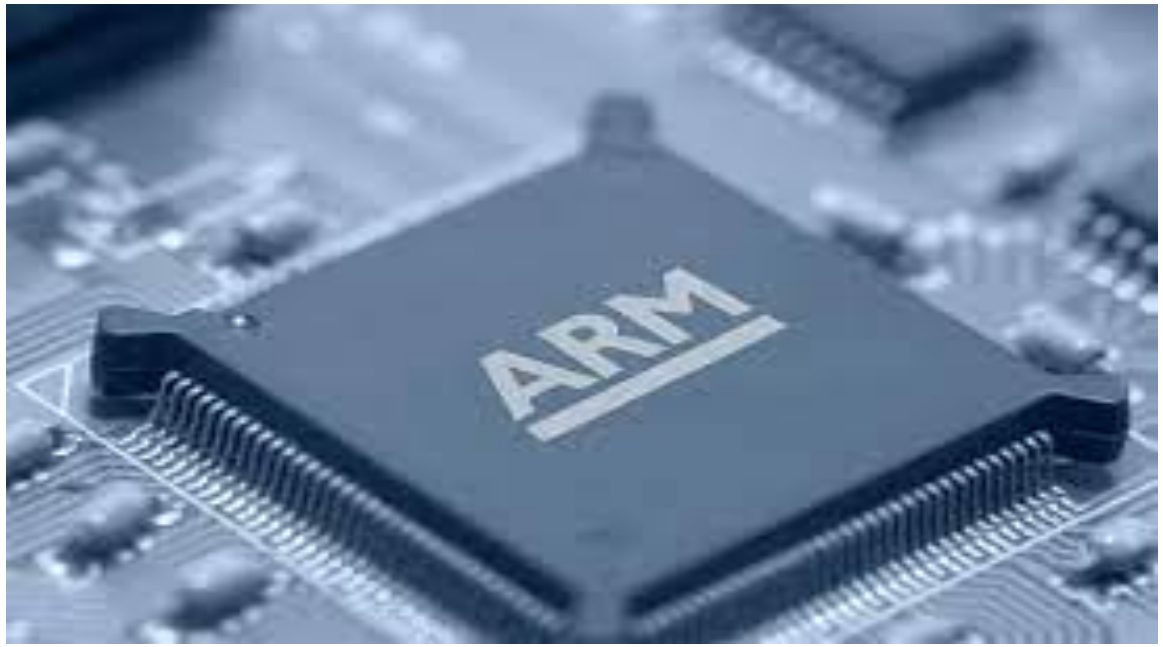
ARM 7 PIPELINING

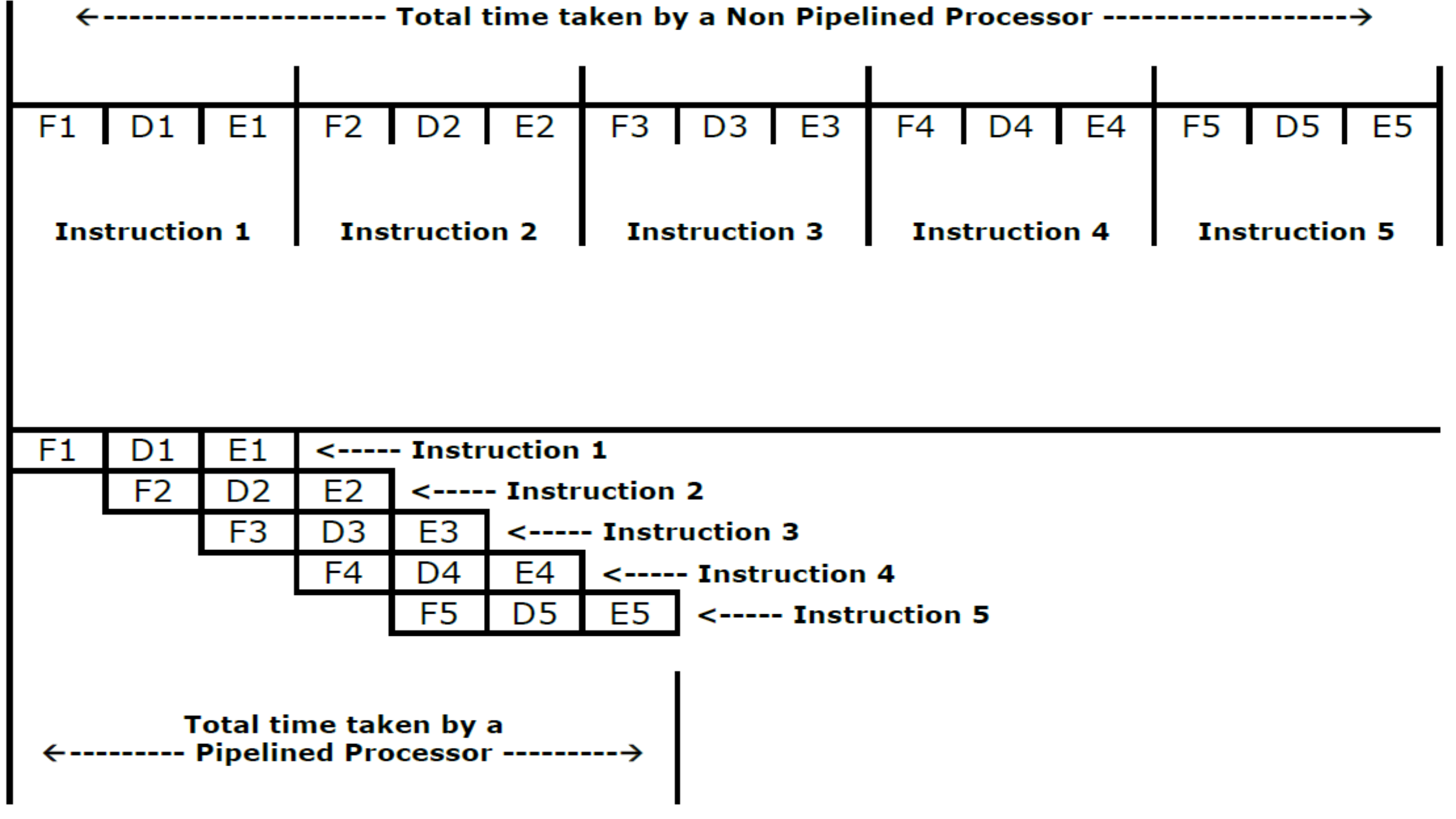


1) Pipelining means **overlapping various stages of an instruction cycle** to improve processor performance

2) ARM7 implements a **3 stage pipeline**.
The instruction cycle is divided into 3 distinct stages of **Fetch, Decode and Execute.**
So when one instruction is being executed, the next one is being decoded and the subsequently next one is being fetched.

3) This means **three different instructions** are being performed at the same time.

4) After the first instruction has been completely executed, every subsequent instruction gets completed in **just one cycle**, which is needed for execution. The fetch and decode operations have already been performed in the background.


←---------- Total time taken by a Non Pipelined Processor ----------→
F1 D1 E1 F2 D2 E2 F3 D3 E3 F4 D4 E4 F5 D5 E5
Instruction 1
Instruction 2
Instruction 3
Instruction 4
Instruction 5
F1 D1 E1 <----- Instruction 1
F2 D2 E2 <----- Instruction 2
F3 D3 E3 <----- Instruction 3
F4 D4 E4 <----- Instruction 4
F5 D5 E5 <----- Instruction 5
Total time taken by a
←--------- Pipelined Processor --------→

5) **Fetch (F):** Here the instruction will be fetched from the memory location specified by PC.
Decode (D): Here the opcode is decoded and control signals are produced for execution.
Execute (E): Here the decoded instruction is executed.

6) ARM7 being a RISC processor, performs superior pipelining due to the following factors
   a) As all instructions are of 32 bits, they take exactly **1 cycle to fetch** through the 32-bit data bus.
   b) As all instructions perform 32-bit data operations, the **execution time** of almost all instructions is the same that is **1 cycle**.
   c) Moreover, as it follows "**load-store**" instruction set model, data operations can only be performed on registers.
   d) So execution of these instructions **does not involve memory access** and hence does not interfere with the fetching process.
   e) **Only load or store instructions need the system bus** for execution and hence fetching has to stop during these instructions.

7) **Pipelining fails during a branch** as it assumes that a program is always executed in a sequential manner. During decode stage if the control unit recognizes that it is a branch instruction, then the subsequent instruction that has been fetched has to be discarded.

8) As this is a 3-stage pipeline, **PC always points two instructions ahead** of the one being currently executed.

	Memory Address	Program Bytes
(PC - 8) →	**0000 2000**	Instruction being executed (4 - Bytes)
	0000 2001	
	0000 2002	
	0000 2003	
(PC – 4) →	**0000 2004**	Instruction being decoded (4 - Bytes)
	0000 2005	
	0000 2006	
	0000 2007	
PC →	**0000 2008**	Instruction being fetched (4 - Bytes)
	0000 2009	
	0000 200A	
	0000 200B	

9) **Higher versions** of ARM processors use **deeper pipelines.**

ARM9 uses **5 stage pipeline.** (Fetch, Decode, Exec, Memory WR, Register WR)

ARM10 uses **6 stage pipeline.** (Fetch, Issue, Decode, Exec, Memory WR, Register WR)

Pipelined Architecture:

- ***ARM 7(3-Stage) :***
  Instruction Fetch (F),
  Instruction Decode (D),
  Execute (E)

- ***ARM9 (5-Stage) :***
  Instruction Fetch (F),
  Instruction Decode (D),
  Execute (D),
  Data Memory access (M),
  Register Write (W)

- ***ARM11(8-Stage) :***
  Two Fetch Stages ,
  A decode Stage,
  An Issue Stage,
  Four Stages for the execution Pipeline
- **ARM Cortex A,R,M** Series bt/w
- 3 to14+ Stages
  With
  Single,Dual,Quadcore,Octacore….

	RISC	CISC
1	Uses **Instructions of a fixed size**	Uses instructions of **variable size**
2	Most instructions take **same time to fetch**.	Instructions have **different fetching times**.
3	Instruction set **simple and small**.	Instruction set **large and complex**.
4	**Less addressing modes** as most operations are register based.	**Complex addressing modes** as most operations are memory based.
5	**Compiler design is simple**	**Compiler design is complex**
6	**Total size of program is large** as many instructions are required to perform a task. This is because instructions are simple.	**Total size of program is small** as few instructions are required to perform a task. This is because instructions are complex & more powerful.
7	Instructions use a **fixed number of operands.**	Instructions have **variable** number of operands.
8	Ideal for processors performing a **dedicated operation**.	Ideal for processors performing a **verity of operations**.
9	Since instructions are simple, they can be decoded by a **hardwired control unit**.	Since instructions are complex, they generally require a **Micro-programmed Control Unit**.
10	Execution speed is **faster** as most operations are register based.	Execution speed is **slower** as most operations are memory based.
11	Since number of cycles per instruction is fixed, it gives a **better degree of pipelining**	Since number of cycles per instruction varies, **pipelining has more bubbles** or stalls.